# Menza

## Zadání

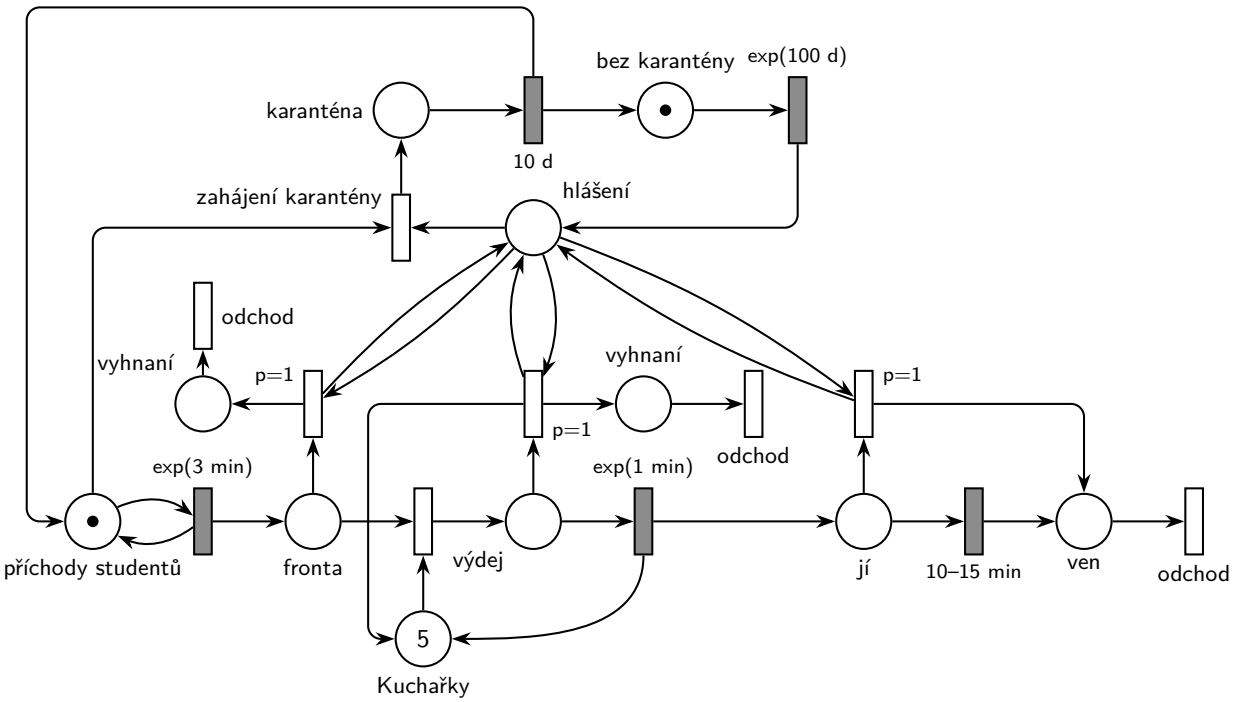
> Do školní menzy přichází studenti v intervalech exponenciálně se středem 3 minuty. Jídlo vydává 5 kuchařek. Doba obsluhy jednoho studenta kuchařkou je exponenciálně se středem 1 minuta. Po obdržení jídla student konzumuje rovnoměrně 10–15 minut. V intervalech exponenciálně se středem 100 dní dostává menza hlášení o žloutenkové karanténě. Po příchodu hlášení se ukončí okamžitě všechny činnosti (výdej, konzumace) a úplně všichni studenti opustí menzu. Po odchodu všech studentů z menzy se zahájí karanténa trvající přesně 10 dní. Příchodem hlášení se zastaví proces příchodu studentů do menzy.
>
> Zadaný systém modelujte formou Petriho sítě s vyznačením všech potřebných parametrů.

Podobné úlohy: Psí salon (vyprázdnění fronty smyčkou s prioritou a zastavení příchodů), Každá 100. transakce vyprázdní frontu, Uhelný mlýn (havárie exp(100 dní)).

## Interpretace

- **Příchody** modeluje místo *příchody studentů* ve smyčce s časovaným přechodem exp(3 min); každé provedení vloží studenta do *fronty*.
- **5 kuchařek** je sklad s 5 značkami: obsazení, exp(1 min) vrací kuchařku. **Konzumace** 10–15 min je časovaný přechod bez omezení: každý jedící student má vlastní odpočet (slajd 138), jedí souběžně. Obsloužený student pak odchází přes *ven* přechodem *odchod*.
- **Hlášení** je proces *bez karantény* → exp(100 d) → *hlášení*. Značka *bez karantény* se vrací až po karanténě, takže hlášení během karantény nepřijde. Zadání to neřeší; interval exp(100 dní) se tu počítá od konce poslední karantény.
- **Všichni ven:** tři vyhazovací přechody s p=1, každý se smyčkou na *hlášení* (hrana tam a zpět). Opakují se, dokud je ve *frontě*, ve *výdeji* nebo mezi *jí* nějaký student. Student vyhozený z výdeje vrací kuchařku. Rozpracovaný výdej i konzumace se tím přeruší, protože okamžitý přechod vezme značku, na které běží odpočet (slajd 139).
- Vyhození studenti opouštějí systém: z fronty a od výdeje přes vlastní místa *vyhnaní* a výstupní přechody *odchod*, jedící přes *ven*.
- **„Po odchodu všech studentů se zahájí karanténa“:** *zahájení karantény* má p=0, takže přijde na řadu, až když žádný vyhazovací přechod (p=1) není proveditelný, tedy když je menza prázdná. Vše proběhne v nulovém čase v okamžiku hlášení. Karanténa trvá 10 d, pak se vrátí značka *bez karantény*.
- **Zastavení příchodů:** *zahájení karantény* vezme značku *příchodů studentů*, tím zanikne odpočet exp(3 min). Po 10 d se značka vrátí. Že se příchody zastaví až s nulovým zpožděním po vyprázdnění, nevadí: mezitím neuplyne žádný čas.

## Petriho síť




## Časté chyby

- Vyhození jedním přechodem s váhou hrany. Váha bere přesný počet, „všichni“ se dělá smyčkou s prioritou.
- Vyhození studenta z výdeje bez vrácení kuchařky: po karanténě by kuchařek bylo méně.
- Karanténa spuštěná přímo hlášením bez priorit. Zadání chce karanténu až po odchodu všech, proto vyhazování p=1 a zahájení karantény p=0.
- Zapomenuté zastavení generátoru studentů: během karantény by se fronta plnila.
- Generátor hlášení ve smyčce bez ohledu na karanténu: druhé hlášení během karantény by spustilo další, souběžnou karanténu.

## SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu.

```
// Menza – model v SIMLIB/C++
#include "simlib.h"
#include <set>

const double DEN = 24 * 60;            // časová jednotka: minuta

Store Kucharky("Kuchařky", 5);         // místo Kuchařky (5 značek)
Histogram Doba("Doba obslouženého studenta v menze [min]", 0, 2, 15);
int karanteny = 0, vyhnani[3] = {0, 0, 0};

enum Stav { FRONTA, VYDEJ, JI };       // ve kterém místě je značka studenta

class Student;
std::set<Student *> vMenze;            // všechny značky v místech fronta, výdej,
 ↪  jí

class Student : public Process {
 public:
  Stav stav = FRONTA;
  Student() { vMenze.insert(this); }
```

```
  ~Student() { vMenze.erase(this); }
  void Behavior() override {
    double prichod = Time;
    Enter(Kucharky, 1);                 // obsazení kuchařky (místo fronta =
 ↪  fronta skladu)
    stav = VYDEJ;                       // místo výdej
    Wait(Exponential(1));               // přechod exp(1 min) ...
    Leave(Kucharky, 1);                 // ... vrací kuchařku
    stav = JI;                          // místo jí
    Wait(Uniform(10, 15));              // přechod 10–15 min: každý jí souběžně
    Doba(Time - prichod);
  }                                     // místo ven + přechod odchod
};

class Prichody : public Event {         // místo příchody studentů ve smyčce s
 ↪  exp(3 min)
  void Behavior() override {
    (new Student)->Activate();
    Activate(Time + Exponential(3));
  }
};
Prichody *prichody;

class Hlaseni : public Process {        // cyklus bez karantény → hlášení →
 ↪  karanténa
  void Behavior() override {
    for (;;) {
      Wait(Exponential(100 * DEN));    // přechod exp(100 d) z místa bez karantény
      // místo hlášení: přechody p=1 vyhazují, dokud je koho; nejdřív fronta,
      // aby uvolněnou kuchařku nedostal nikdo z fronty
      for (int s : {FRONTA, VYDEJ, JI}) {
        std::set<Student *> kopie = vMenze;
        for (Student *st : kopie)
          if (st->stav == s) {
            if (s == VYDEJ)
              Kucharky.Leave(1);        // vyhnaný od výdeje vrací kuchařku
            vyhnani[s]++;
            st->Cancel();               // přeruší Wait a odstraní značku (odchod)
          }
      }
      // zahájení karantény (p=0): menza je prázdná, bere značku příchodů
      prichody->Passivate();
      karanteny++;
      Wait(10 * DEN);                   // místo karanténa, přechod 10 d
      prichody->Activate(Time + Exponential(3)); // vrací značku příchodů
 ↪  studentů
    }
  }
};

int main() {
  Init(0, 1000 * DEN);                  // modelový čas v minutách: 1000 dní
  prichody = new Prichody;
  prichody->Activate();
  (new Hlaseni)->Activate();
  Run();
  Print("Karantény: %d\n", karanteny);
  Print("Vyhnaní z fronty: %d, od výdeje: %d, od jídla: %d\n",
        vyhnani[FRONTA], vyhnani[VYDEJ], vyhnani[JI]);
  Kucharky.Output();
  Doba.Output();
}
```

| Petriho síť | SIMLIB |
|---|---|
| *příchody studentů* ⇄ exp(3 min) | událost Prichody, která se znovu plánuje za Exponential(3) |
| značka (student) | proces Student; místo, ve kterém je, udává stav |
| *fronta* + obsazení kuchařky | Enter(Kucharky, 1) (fronta je vnitřní fronta skladu) |
| *Kuchařky* (5 značek) | Store Kucharky("Kuchařky", 5) |
| *výdej* → exp(1 min) se sloučeným uvolněním | Wait(Exponential(1)); Leave(Kucharky, 1); |
| *jí* → 10–15 min | Wait(Uniform(10, 15)) |
| *ven* + *odchod* | konec metody Behavior() |
| *bez karantény* → exp(100 d) → *hlášení* | proces Hlaseni: Wait(Exponential(100 * DEN)) |
| vyhazovací přechody p=1 se smyčkou na *hlášení* | cyklus přes vMenze: Cancel() studenta, u výdeje navíc Kucharky.Leave(1) |
| *vyhnaní* + *odchod* | Cancel() (proces skončí) |
| *zahájení karantény* (p=0) bere značku příchodů | prichody->Passivate() po vyhnání všech |
| *karanténa* → 10 d vrací značky | Wait(10 * DEN); prichody->Activate(...), pak další průchod cyklem |

**Kontrola simulací.** Petriho síť: *Kuchařky* + *výdej* = 5, *příchody studentů* + *karanténa* = 1, *bez karantény* + *hlášení* + *karanténa* = 1 (make check). Hlášení chodí jednou za 100 dní, takže i 300 000 událostí (asi 200 dní) dá jen 0–3 karantény a fronta k 5 kuchařkám je v tu chvíli skoro vždy prázdná. Vyhazování proto ověřil zrychlený běh (hlášení po exp(1 h), karanténa 1 h, příchody po exp(0,15 min)): všechny tři vyhazovací přechody se provedly tisíckrát a invarianty platily. SIMLIB model: za 1000 dní proběhlo 7 karantén (očekává se asi 1000 / 110 ≈ 9), vyhnáno bylo 31 jedících, 2 studenti od výdeje a nikdo z fronty. Kuchařky byly v průměru obsazené 0,31 (teoreticky 1/3 studenta za minutu × 1 min × 100/110 času mimo karanténu ≈ 0,30), obsloužený student strávil v menze v průměru 13,5 minuty (1 min výdej + 12,5 min jídlo).

## Původ a zdroje

**Původ:** půlsemestrálky 2014–19 (přehled zadání), úloha Z9.

**Původní znění:**

> Do školní menzy přichází studenti v intervalech exp(3 min). Jídlo vydává 5 kuchařek. Doba obsluhy jednoho studenta kuchařkou se řídí exp(1min). Po obdržení jídla student konzumuje po dobu 10-15 minut rovnoměrně. V intervalech exp(100 dní) dostává menza hlášení o žloutenkové karanténě. Po příchodu hlášení se ukončí okamžitě všechny činnosti (výdej, konzumace) a úplně všichni studenti opustí menzu. Po odchodu všech studentů z menzy se zahájí karanténa trvající 10 dní. Příchodem hlášení se zastaví proces příchodu studentů do menzy.
>
> Zadaný systém modelujte formou Petriho sítě s vyznačením všech potřebných parametrů.

**Zdroje:**

- Fitcheats/Půlsemestrálky/Zadání z minulých let/Přehled zadání 2014-19.pdf, str. 3 (zadání)
- Fitcheats/Půlsemestrálky/FIT - Příprava 2021-22.pdf, str. 13 (studentské řešení, 0 pro / 2 proti)
- Fitcheats/Příprava na zkoušku/luppo - Tabule.png (studentské řešení na tabuli)

**Existující řešení:**

- **Příprava 2021/22 (str. 13, 0 pro / 2 proti):** obrácené značení (šrafovaně okamžité přechody). Výdej je jediný časovaný přechod exp(1m) se vstupy z fronty a z *kuchařek* (kuchařku hned vrací), takže obsluhovaní studenti zůstávají ve frontě a vyhazovací přechod P=2 je bere

spolu s čekajícími. Konzumující vyhazuje P=1. Chyby, které vytkly i komentáře: příchody studentů se nezastaví a karanténa 10d startuje přímo z generátoru hlášení, ne až po vyprázdnění. Studenti nemají výstupní přechod.

- **Tabule luppo:** *Studenti* → Exp(3min) → *Příchod* → obsazení kuchařek (5) → *Obsluha* → Exp(1min) → *Žere* → Rav(10-15) → *Odchod*. *hlášení* má smyčky na vyhazovací přechody z fronty (p=2), z obsluhy (vrací kuchařku) i z *Žere*. Časovač 10dní je přímo na místě *hlášení*. Hrana, která by brala značku generátoru studentů, na fotce není vidět.